

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

You are tasked with designing a program to help digitize scans of Medieval English manuscripts. A character-recognition program has already converted the scans into a string X that is composed of n characters.

Medieval manuscripts were often written uncapitalized and *without any spaces*, or with spaces that are hard for the recognition program to identify, so X contains no spaces between words and only lower-case letters. The recognition is also far from perfect, and some of the letters have been misread. This causes some words to become misspelled. We have a procedure $\text{ISWORD}(w)$, which returns **true** if a string w is a recognized Medieval English word and **false** otherwise. Assume that ISWORD works in $\Theta(1)$ time regardless of the length of w .

We would like to insert spaces into appropriate places in X so that the total number of characters in misspelled words is minimized. We are not interested in correcting the misspelled words yet.

Develop an efficient algorithm to report the minimum number of characters in misspelled words, and the indices where spaces should be inserted.

Example

Input: $X = \text{allworkdmdnoplaymakesjackqdullbpy}$ where $n = 33$.

Output: “*all work dmd no play makes jack q dull bpy*” is an optimal solution with a cost of $3 + 1 + 3 = 7$, since all words except *dmd*, *q*, and *bpy* are correctly spelled in this solution.

Presentation 1 Solution

1: The “last” decision

The last decision is choosing the *final* position in string X to insert a space, that leads to the minimum number of characters in misspelled words.

2: Define subproblems

Let $dp[i]$ represent the minimum number of characters in misspelled words if we consider $X[1\dots i]$: the substring of X ending at position i .

Let $prev[i]$ be the index of the last space inserted before position i , in the optimal solution that minimizes the number of characters in misspelled words in $X[1\dots i]$.

3: Develop recursive formulation to solve subproblems

$$dp[i] = \begin{cases} 0 & \text{if } i = 0 \\ \min_{j < i} (dp[j] + \text{Cost}(X[j+1\dots i])) & \text{otherwise} \end{cases} \quad (1)$$

$$prev[i] = \begin{cases} \text{NIL} & \text{if } i = 0 \\ \text{argmin}_{j < i} \{dp[j] + \text{Cost}(X[j+1\dots i])\} & \text{otherwise} \end{cases} \quad (2)$$

where $\text{Cost}(X[j+1\dots i])$ is 0 if $X[j+1\dots i]$ is a meaningful word (i.e., $\text{ISWORD}(X[j+1\dots i]) = \text{true}$) and is equal to the length of the string $X[j+1\dots i]$ otherwise.

We can implement this formulation in a bottom-up manner as follows:

Algorithm 1 Minimize Misspelled Characters

```

1: function MINIMIZECHARS( $X$ )
2:    $dp \leftarrow$  Array of length  $n$ 
3:    $prev \leftarrow$  Array of length  $n$ 
4:   for  $i \leftarrow 1$  to  $n$  do
5:      $dp[i] \leftarrow \text{RECURRENCERELATION}(X, dp, i)$      $\triangleright$  Compute value using recurrence relation
6:      $prev[i] \leftarrow \text{DETERMINEPREVIOUS}(X, dp, i)$      $\triangleright$  Determine the previous space position
7:   end for
8:   return RECONSTRUCTSOLUTION( $X, prev, n$ )           $\triangleright$  Reconstruct the optimal solution
9: end function

```

Here, the procedure `RECURRENCERELATION` implements Equation (1) to calculate $dp[i]$, and `DETERMINEPREVIOUS` determines the index of the previous space that leads to the optimal solution for $prev[i]$.

The procedure `RECONSTRUCTSOLUTION` uses the table $prev$ to construct the solution with the minimum number of meaningless characters.

4: Reconstruct the optimal solution

To determine the optimal spacing, we backtrack from the string where we've reduced the most misspellings stored in $choices[i]$.

Algorithm 2 Reconstruct String with Minimal Misspellings

```

1: function RECONSTRUCTSOLUTION( $X, prev, n$ )
2:   Initialize empty string  $S$ 
3:    $i \leftarrow n$ 
4:   while  $i > 0$  do
5:      $X[prev[i] \dots i] \cap S$ 
6:      $i \leftarrow prev[i]$ 
7:   end while
8:   return  $S$ 
9: end function

```

5: Show execution on an example

Consider the shorter example $X = \text{"yesxkno"}$ where $n = 7$.

The following tables should be computed progressively:

i	0	1	2	3	4	5	6	7
$X[i]$	—	y	e	s	x	k	n	o
$dp[i]$	0	1	2	0	1	2	3	2
$prev[i]$	NIL	0	0	0	3	3	3	5

(3)

Since the value of $prev[n]$ is 5, we can reconstruct the solution recursively as follows:

- $i = n = 7$. $prev[i] = 5$ thus append $X[6...7] = \text{"no"}$ to the solution
- $prev[5] = 3$, thus append $X[3...5] = \text{"xk"}$ to the solution
- $prev[3] = 0$, thus append $X[0...3] = \text{"yes"}$ to the solution
- now $prev[i] = NIL$.
- The reconstructed solution will be "yes xk no" with a total cost of 2 for the 2-letter misspelled word "xk".

6: Prove Optimal Substructure

We want to show that if a spacing is optimal for $X[1...i]$ with $i < n$, then it is optimal for $X[1...i-1]$. The problems are independent of each other because an optimal spacing for substring $X[1...i]$, can not involve inserting any spaces in the region $X[i+1...n]$.

Given: Let S be an optimal solution for the problem for $X[1...i]$, where $i < n$. We can decompose S into a sequence of m individual words, $X[1...w_1] + X[w_1+1...w_2] + \dots + X[w_{m-2}+1...w_{m-1}] + X[w_{m-1}+1...w_m]$ (where $w_m = i$). We will call the number of characters in misspelled words in S , C_S . If we were to remove the last character in the sequence, we would get a solution $S' = X[1...w_1] + X[w_1+1...w_2] + \dots + X[w_{m-2}+1...w_{m-1}-1]$. If $X[w_{m-1}+1...w_m]$ was misspelled, the number of characters in misspelled words in S' (we will call it $C_{S'}$) will be $C_S - 1$. If $X[w_{m-1}+1...w_m]$ was not misspelled, $C_{S'} = C_S$.

Claim: S' is an optimal solution for the spacing problem for the substring $X[1...i-1]$.

Proof by Exchange of Arguments: Assume that the claim is **false**. This means that S' is **not** an optimal solution to the spacing problem for the string $X[1...i-1]$. This means that there must be an alternative optimal solution S'' that contains fewer characters in misspelled words than S' . ($C_{S''} < C_{S'}$). Now, let's reconstruct the optimal subproblem by appending $X[i]$, creating the solution S^* .

If $X[w_{m-1}+1...w_m]$ was misspelled, $C_{S^*} = C_{S''} + 1 < C_S = C_{S'} + 1$.

If not, $C_{S^*} = C_{S''} < C_S = C_{S'}$. In either case, we have used S'' to construct a solution S^* with a lower cost than S . However, S is optimal, so the existence of S^* is a contradiction. Therefore, no S'' can exist: S' is the optimal solution for the subproblem $X[1...i-1]$.

7: Runtime

To compute the DP table:

- We iterate through each character position i to determine $dp[i]$ and $prev[i]$.
- For each position i , we potentially need to check all previous positions to find the minimum $dp[j]$ for $j < i$ and to calculate the cost of forming a word from $j + 1$ to i .

Thus, for each i , the worst-case scenario involves iterating through i previous positions.

Therefore, computing the DP table will take at most

$$T(n) = \sum_{i=1}^n \Theta(i) = \Theta(n^2) \tag{4}$$

time.

Since reconstructing the optimal solution makes one pass over the array $prev$, it takes $\Theta(n)$ time. Thus, the total worst-case runtime of the algorithm is $\Theta(n^2) + \Theta(n) = \Theta(n^2)$.

Presentation 2

You are creating a program to manage the radar system on a space probe, which will be performing a close flyby of Mars. The probe flies over the Martian surface in a straight line at a rate of 1 km per second, passing over n sectors that are each 1 km wide. The radar system can scan these sectors, but needs to recharge after doing so, during which time it is inoperative:

- A **surface** scan recharges for 1 second and cannot scan the very next sector.
- A **medium**-depth scan recharges for 2 seconds and misses the next 2 sectors.
- A **deep** geological scan recharges for 10 seconds and misses the next 10 sectors.

The science team has assigned point values estimating how much information they can learn from scanning each sector, and what kind of scan will be needed to get that information (as indicated by the recharge time). Your task is to determine what sectors to scan to maximize the information gained. (The probe can wait longer than the recharge time to scan more important sectors.)

The array V contains the value estimates of each sector ($V[i]$ is the value of scanning Sector i). The array S contains the recharge times ($S[i]$ is the number of sectors that will pass by before the probe can scan again, after scanning Sector i - either 1, 2, or 10).

Develop an efficient algorithm to report the maximum possible information value, and a list of sector indices to scan to produce it.

Example

Input:

- $n = 5$
- $S = \{1, 2, 1, 10, 1\}$
- $V = \{5, 8, 3, 10, 7\}$

Output: [1,4]

Scan Sector 1 to gain $V[1] = 5$, recharge when passing over Sector 2. Wait as we pass over Sector 3. Scan Sector 4 to gain $V[4] = 10$, recharge for 10 seconds (during which we will pass over Sector 5 and leave Mars's atmosphere), with a total point value of $5+10 = 15$.

Presentation 2 Solution

1: The “last” decision

A typical way to define $r[k]$ is the maximum value achievable for sectors $1 \dots k$. However, the recharge constraint of the problem is applied in the direction of increasing time, i.e. given that we scanned sector t , then some number of sectors that appear after t cannot be scanned. This suggested relation goes in the opposite direction, i.e. it relies on solutions to previous sectors. So, we will want to start at the end of the array and work backwards. The “last” decision we want to make, is whether or not we want to scan the *first* sector.

2: Define subproblems

Let $B[i]$ represent the maximum amount of information available if considering sectors from i to n .

Let $chosen[i]$ represent whether or not we will be scanning Sector i in our optimal solution that maximizes the amount of information available from sectors i to n .

3: Develop recursive formulation to solve subproblems

$$B_i = \begin{cases} 0 & \text{if } i > n \\ v_n & \text{if } i = n \\ \max(v_i + B_{i+S_i}, B_{i+1}) & \text{if } 1 \leq i < n \end{cases}$$

In this case, B_I represents the maximum inspection value if boxes $i \dots n$ are in consideration. When passing over Sector i , either we can choose to scan it, or not scan it. If we choose to scan it, then we know the next $S[i] - 1$ sectors cannot be scanned. If we choose not to scan it, our only constraint for selecting the next sector to scan is that it must be after Sector i .

We can implement this formulation in a bottom-up manner as follows:

Algorithm 3 Radar Scanning Problem

```

1: procedure MAX-INSPECTION-VALUE( $S, V, n$ )
2:    $B \leftarrow$  array of size  $(n + 10)$ 
3:    $B[n + 1..n + 10] \leftarrow 0$  ▷ First Case
4:    $B[n] \leftarrow V[n]$  ▷ Second Case
5:    $chosen \leftarrow$  array of size  $n$ 
6:
7:   for  $i = n - 1$  to  $1$  do
8:      $B[i] \leftarrow \max(B[i + 1], v[i] + B[i + S[i]])$  ▷ Third Case
9:     if  $B[i] = B[i + 1]$  then
10:       $chosen[i] \leftarrow \text{False}$ 
11:     else
12:       $chosen[i] \leftarrow \text{True}$ 
13:     end if
14:   end for
15:
16:   return RECONSTRUCTSCANS( $S, V, n, B, chosen$ ) ▷ Solution Construction
17: end procedure

```

4: Reconstruct the optimal solution

Once we know what sectors will be scanned and which ones will not (as stored in $chosen$), we can simply advance through the arrays V and S , appending each scanned sector to our list:

Algorithm 4 Reconstruct Radar Itinerary

```
1: procedure RECONSTRUCTSCANS( $S, V, n, B, chosen$ )
2:    $finalScans \leftarrow$  empty array
3:    $trav = 1$ 
4:   while  $trav \leq n$  do
5:     if  $chosen[trav]$  then
6:        $finalScans.append(trav)$ 
7:        $trav \leftarrow trav + S[trav]$ 
8:     else
9:        $trav \leftarrow trav + 1$ 
10:    end if
11:  end while
12:  return  $B[1], finalScans$ 
13: end procedure
```

5: Show execution on an example

Consider the example $S = \{1, 2, 1, 10, 1\}$, $V = \{5, 8, 3, 10, 7\}$ with $n = 5$, as given above.

The following table should be computed from end to beginning:

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$S[i]$	1	2	1	10	1										
$V[i]$	5	8	3	10	7										
$B[i]$	15	10	10	10	7	0	0	0	0	0	0	0	0	0	0
$chosen[i]$	<i>Y</i>	<i>N</i>	<i>N</i>	<i>Y</i>	<i>Y</i>										

(5)

To reconstruct the solution, we will progress from the beginning of the array. We will scan Sector 1 for 5 points, and then advance 1 space. We will not scan Sector 2, and again advance 1 sectors. We will scan Sector 4 for 10 points, and then advance 10 sectors, skipping over the **true** value in $chosen[5]$. Since this puts $i > n$, we will stop processing the array and return a scan target list of $[1, 4]$ with a total information gain of 15.

6: Prove Optimal Substructure

Since the construction of the arrays B and $chosen$ determines whether or not to scan Sector i based only on the values for Sectors $i \dots n$, the algorithm is correct if the problem possesses the property of optimal substructure.

Working back-to-front, the subproblems are independent of each other since the recharge constraint only moves forward in time, i.e. choosing Sector i does not restrict future Sectors j s.t. $j < i$ from being chosen.

Given: Let L be an optimal solution for an instance of the problem for sectors $i \dots n$. Assume that sector i is scanned in the optimal solution. If not, we will keep incrementing i until a subproblem of size $j > i$ is chosen where Sector j is in the solution. Then, $L = v[i] + \dots + v[j]$ where $i \leq j \leq n$.

Based on the constraints of the problem, we know Sectors $i + 1 \dots i + s[i] - 1$ do not exist in L . Therefore, we can remove Sector i to obtain solution

$$L' = v[k] + \dots + v[j]$$

where $i + S[i] \leq k \leq j$.

Claim: L' is an optimal solution to an instance of the problem for Sectors $i + S[i] \dots n$.

Proof by Exchange of Arguments: Assume that our claim is **false**. This means that L' is **not** the optimal solution. Then, there must be an alternative optimal solution L'' that is feasible and provides a higher information value than L' . Now, reconstruct the optimal solution for the original problem in the following way:

$$L^* = L'' + v[i] > L' + v[i] = L$$

Here, L^* is a feasible solution for the original problem because Sector k has been guaranteed not to violate the recharge constraints. L^* also provides a higher information value than the given optimal solution L . This creates a contradiction, as L was assumed to be optimal. Therefore, L'' cannot be constructed, and L' must be an optimal solution for the problem for Sectors $i \dots n$.

7: Runtime

We need to consider

1. The number of subproblems
2. The amount of work done on each subproblem

There are n total subproblems being solved, and each problem takes $\Theta(1)$ time to calculate. Therefore, it takes $\Theta(n)$ time to calculate the maximum information gain. The construction of the final scan sequence in the solution also takes at worst $\Theta(n)$ time, because it does one traversal through the table of subproblems. Therefore, the total runtime of the algorithm is $\Theta(n) + \Theta(n) = \Theta(n)$.